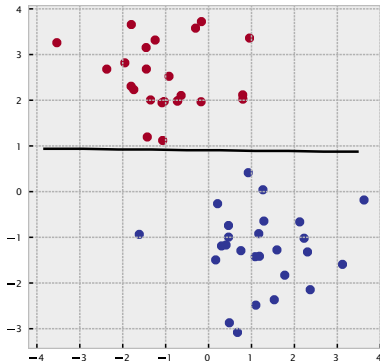


# Support vector classifiers

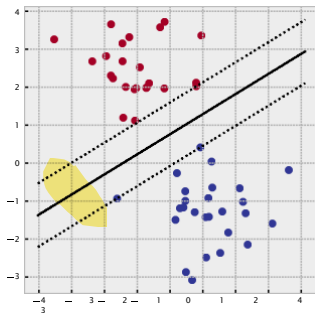
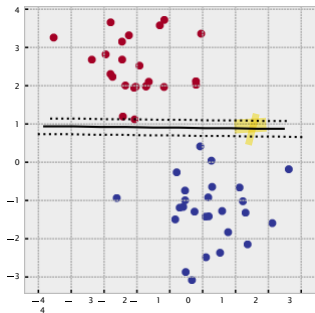
# geometric view

- geometrically, a linear classifier can be interpreted as separating the vector space into two regions using a line (plane, hyperplane)



# margin of separation

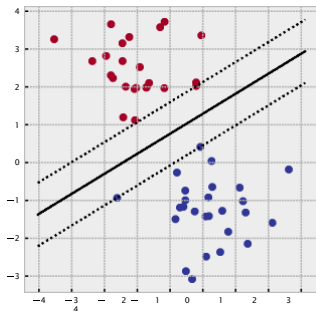
- the **margin**  $\gamma$  denotes how well  $w$  separates the classes:



- $\gamma$  is the shortest distance from the separator to the nearest training instance

# support vector classifiers

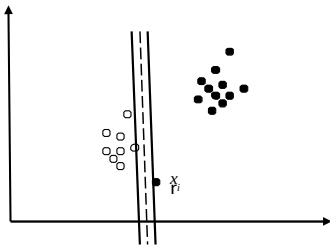
- **support vector machines (SVM)** or support vector classifiers (SVC) are linear classifiers constructed by selecting the  $w$  that maximizes the margin



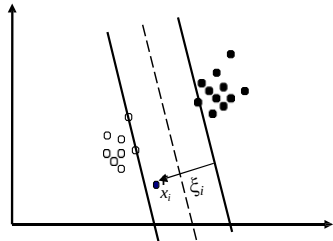
- note: the solution depends only on the borderline  
examples: the **support vectors**

# soft-margin SVMs

- in some cases the dataset is inseparable, or nearly inseparable
- **soft-margin SVM**: allow some examples to be disregarded when maximizing the margin



A) Hard Margin SVM



B) Soft Margin SVM

## stating the SVM as an objective function

- the hard-margin and soft-margin SVM can be stated mathematically in a number of ways
- we'll skip the details, but it can be shown (see Daumé's book) that the soft-margin SVM can be stated as minimizing

$$\frac{C}{N} \cdot \sum_{i=1}^N \text{Loss}(\mathbf{w}, \mathbf{x}_i, y_i) + \frac{1}{2} \|\mathbf{w}\|^2$$

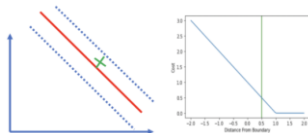
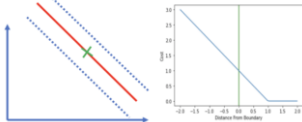
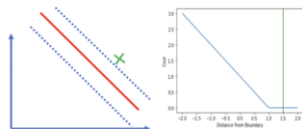
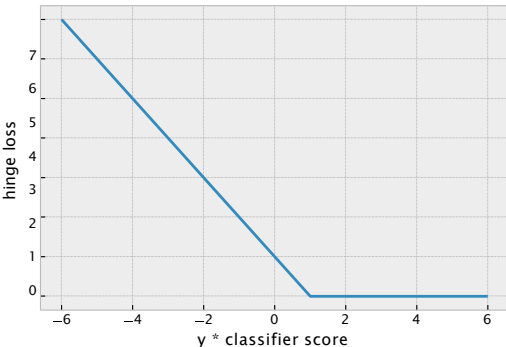
where

$$\text{Loss}(\mathbf{w}, \mathbf{x}, y) = \max(0, 1 - y \cdot (\mathbf{w} \cdot \mathbf{x}))$$

is called the **hinge loss**

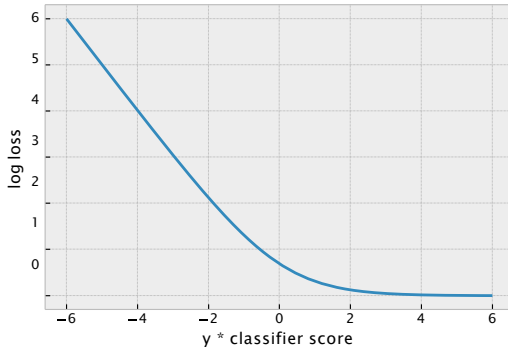
# plot of the hinge loss

$$\text{Loss}(\mathbf{w}, \mathbf{x}, y) = \max(0, 1 - y \cdot (\mathbf{w} \cdot \mathbf{x}))$$



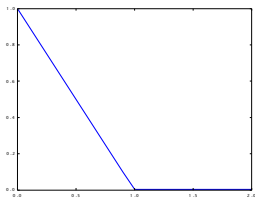
<https://programmatically.com/understanding-hinge-loss-and-the-svm-cost-function/>

## recap: plot of the log loss / binary cross-entropy

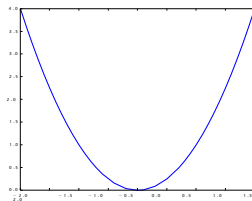




the **SVC** has a **convex objective function**



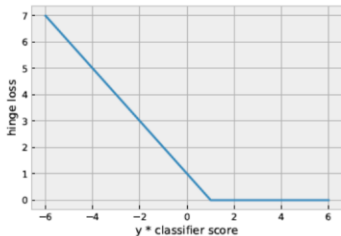
+



# Example: Margin

# SVM objective function

What happen if we increase/decrease the value of C?



$$\frac{C}{N} \cdot \sum_{i=1}^N \text{Loss}(\mathbf{w}, \mathbf{x}_i, y_i) + \frac{1}{2} \|\mathbf{w}\|^2$$

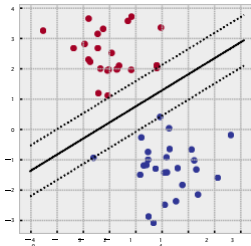
where

$$\text{Loss}(\mathbf{w}, \mathbf{x}, y) = \max(0, 1 - y \cdot (\mathbf{w} \cdot \mathbf{x}))$$

# the dual form of the SVC

- it can be shown that the SVC can be rewritten as:

$$\mathbf{w} \cdot \mathbf{x} = \sum_{i=1}^N \alpha_i \cdot \mathbf{x}_i \cdot \mathbf{x}$$



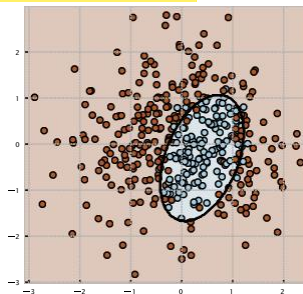
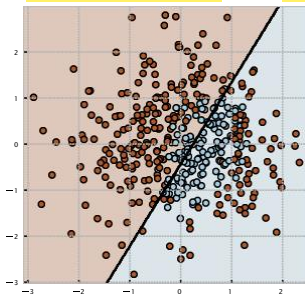
- note: only need support vectors, all other  $\alpha_i$  are zero

# the nonlinear SVC

- we can generalize the SVC by replacing the dot product with a **kernel function**:

$$\sum_{i=1}^N \alpha_i \cdot K(x_i, x)$$

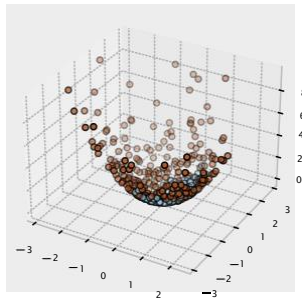
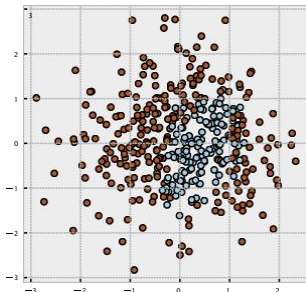
- with a **nonlinear kernel**, our **classifier is not linear** any more!



- typically much **slower to train**, but can **deal with more complex datasets**

# what are these kernels?

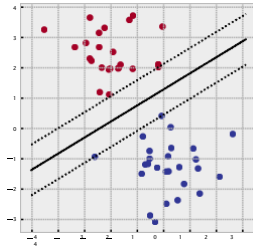
- a kernel **implicitly** computes the dot product **in a more high-dimensional feature space**
- mathematically, a kernel is like first mapping the features into a new space, and then computing the dot product



# Example: NonLinear

# Summary

the **support vector classifier** is based on the idea of **maximizing the margin** between the classes



in the simplest case it is a **linear classifier**

but it can be **made nonlinear** by using **kernels**